

Implement an acceptance test

In this exercise you need to implement acceptance test for a device that uses Modbus ASCII-protocol. The device has only input and holding registers and implements a subset of Modbus function codes for accessing the registers.

The device address is: 17.

The supported function codes are:

- 3 – read holding registers
- 4 – read input registers
- 6 – write holding register
- 16 – write multiple holding registers

Valid register address ranges:

- Holding registers: 10 – 20 (inclusive)
- Input registers: 1 – 9 (inclusive)

Both input and holding registers are addressed by 16-bit address and there is a separate address space for both register types. The device implements holding registers in the range 10 – 20 (inclusive) and input registers in the range 1 – 9 (inclusive). These addresses are internal addresses where the numbering starts from zero. Typically, Modbus libraries use internal addresses to access registers.

There is also Modicom addressing where numbering starts from one and the addresses are prefixed to indicate the type of address (coils, input registers, holding registers, etc.). Modicom address is converted to internal address by removing the prefix and subtracting one from the address. In this exercise we are not using Modicom addressing.

Modbus specification (<https://www.modbus.org/file/secure/modbusprotocolspecification.pdf>) has detailed description of each function code and the expected behavior of the device in case of errors and/or invalid requests. There is a generic flow chart for processing received Modbus frames and a more detailed flow chart for each function code.

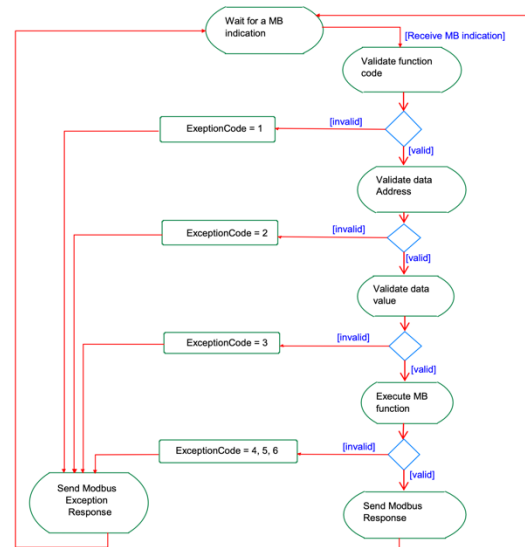


Figure 1 Generic Modbus function code processing

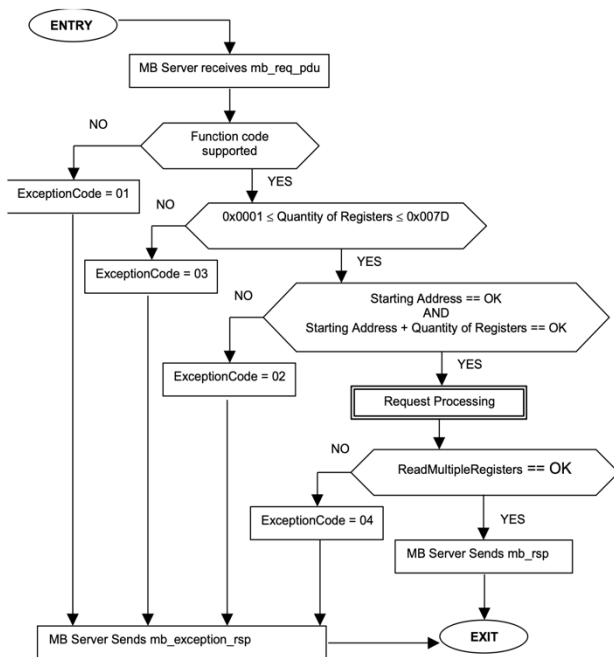


Figure 13: Read Holding Registers state diagram

Figure 2 Function code specific flow chart

The detailed flow chart shows function code specific details for validating function code parameters.

MinimalModbus, a Python library, implements a subset of Modbus function codes. It implements all function codes needed in this test and reports error by throwing exceptions. Any other library that supports Modbus ASCII can also be used. MinimalModbus requires PySerial which you should already have installed in the previous exercises. See: <https://minimalmodbus.readthedocs.io/> for more information.

```

import minimalmodbus

if __name__ == '__main__':
    mb = minimalmodbus.Instrument("/dev/cu.usbmodem12301", 17, mode=minimalmodbus.MODE_ASCII)
    print(mb.read_registers(2, 2, 4))
    mb.write_register(13, 5665, functioncode=16);
    print(mb.read_registers(13, 1, 3))
  
```

Figure 3 Example of minimalModbus usage

Requirements

The goal of this exercise is to implement an acceptance test that verifies that device follows Modbus specification:

- Unsupported function codes are reported with an exception code
- All supported function codes work as specified
- Parameters are validated and errors reported. You should test:
 - Valid values with different combinations
 - Invalid values, for example:
 - Invalid register number
 - Invalid range (one end of range out of specs, both ends out of specs, etc.)
- Reading from and writing to registers works as specified
- Test also that you can read back values that you have written to registers

Keywords and test cases must be named descriptively so that top level tests can be understood without reading the comments.